GenAI Assignments

Over seven sessions, Foundations, prompting, RAG, agents, agent architectures, MCP, and the audio/visual stack were covered. Each session ended with a bit of homework. Now you put the pieces together into one working thing.

Plan for roughly 5 to 7 days of focused effort. You'll get more calendar time than that, because you have project work and other commitments running alongside this, and the team will confirm the final dates. Use the extra days for the demo recording and the write-up, not for starting late.

How this works

Pick one of the three problem statements below. Just one, not all three. Whichever you choose, the same requirements, deliverables, and rubric apply, and they're listed once right after the options.

Each option is built so you can't finish it without touching all seven sessions.

Option 1: Meeting-notes copilot

You sit through back-to-back meetings and lose half of what got decided. Build a tool that takes a 2- to 3-minute audio recording of a meeting and turns it into something useful and searchable. Record the audio yourself. A standup, a design discussion, or even you reading out a fake meeting will do.

What it must do:

- Transcribe the recording to text.
- Summarize the transcript into a **structured JSON** object. At a minimum, that's a short summary plus a list of action items, each with an owner, a task, and a due date where one is mentioned.
- Store those summaries in a vector store so meetings build up over time. Seed it with at least three or four meetings (re-record short clips or paste in a few transcripts) so retrieval has something to work with.
- Let a user ask follow-up questions across past meetings, like "What did we decide about the billing migration?" and answer **only from retrieved context**, citing the meeting it came from.
- Include an agent that can take one real action through an MCP tool. Creating a calendar entry or writing an action-items file to disk, for example

---

Option 2: Support-ticket triage bot

Picture a support inbox like Flipkart's or Zomato's. A steady stream of tickets comes in, and most look a lot like ones the team has already solved. Build a bot that helps an agent triage a new ticket faster.

What it must do:

- Start from a small corpus of past tickets. Write or generate around 15 to 20, and make them yours rather than a generic placeholder set.
- For a new incoming ticket, retrieve the most similar past tickets with RAG.
- Classify and **route** the ticket by category (billing, delivery, technical, and so on) using a router pattern. Different categories should take visibly different paths.
- Draft a **structured reply** as JSON: category, suggested response, confidence, and the IDs of the similar tickets you matched.
- Accept a **voice note** as the input for a new ticket. Transcribe it, then run the same pipeline.
- Use an MCP tool to "file" the ticket. Append it to a tickets store, write a JSON record, something along those lines.

Option 3: Doc-to-podcast pipeline

Take a document and turn it into a short narrated explainer, the kind of thing you'd actually listen to on a commute. This one leans on planning and self-correction.

What it must do:

- Run RAG over a document of your choice. A README, a policy doc, a paper, anything real rather than lorem ipsum.
- Have an agent **plan** the script in steps (plan-and-execute). Outline first, then expand each section using retrieved content.
- Add a **critic/reflection** step that reviews the draft against the source. It flags any claim the document doesn't support, and the agent revises before moving on.
- Run the final script through **TTS** to produce narrated audio.
- Use an MCP tool to save the audio file and the final transcript locally, side by side.

What every submission must include
Whichever option you pick:

- **It runs.** A README with clear setup and run steps, plus a `requirements.txt` or equivalent. Assume I'll clone it on a fresh machine.
- **Your own data.** Your own recording, your own ticket corpus, your own document. Generic or obviously model-generated placeholder data is the fastest way to make a submission look thin.
- **Optional Per-stage timing and token counts.** Log the latency of each stage (`stt_ms`, `llm_ms`, `tts_ms`, `total_ms`, the same idea as the Session 7 demo) along with the token counts for your model calls. We spent time on cost and context budgets in Session 1, so show you're watching them.
- **Grounded generation.** Your system prompt has to constrain the model to the retrieved context, and the system should be willing to say "I can't find this in the provided context" instead of inventing an answer.
- **Safety on actions.** Anything that writes, sends, or deletes goes through a real tool call, with a `MAX_ITERATIONS` guard on the agent loop. For anything irreversible, a human-in-the-loop confirmation is a nice touch.

Map it back to the sessions

Before you submit, walk through this list. If you can't point to where each item lives in your code, you have a gap to close.

- ☐ **Session 1 (Foundations).** API key in an env var, never committed. A `messages` array you manage yourself, and a temperature you chose on purpose.
- ☐ **Session 2 (Prompting).** Structured or JSON output, plus at least one of few-shot or chain-of-thought where it actually earns its place.
- ☐ **Session 3 (RAG).** Chunking, embeddings, a vector store, top-k retrieval, and answers cited from context.
- ☐ **Session 4 (Agents).** A tool-calling loop (Thought, Action, Observation), correct `tool_call_id` handling, every tool call logged, and an iteration cap.
- ☐ **Session 5 (Architectures).** A named pattern you picked deliberately (router, sequential pipeline, plan-and-execute, or reflection/critic), with a sentence on why.
- ☐ **Session 6 (MCP).** At least one tool or resource exposed or consumed over MCP, not just a hard-coded function call.
- ☐ **Session 7 (Audio/Visual).** Speech-to-text or text-to-speech wired into the flow, with per-stage latency logged.

What to submit

1. A **Git repository** (GitHub or similar) with the code, the README, and your data files.
2. A **3 to 5 minute screen recording** where you run the thing end to end and narrate what's happening. This is the single most useful artifact for review, and it takes about ten minutes to make.
3. A **Design Note of around 300 words** that answers three things in your own words: which architecture pattern you chose and why; one failure you hit while building and how you debugged it; and one tradeoff you made (RAG versus longer context, batch versus real-time, high k versus low, whatever applies to you).
4. A **small evaluation set**: 5 to 10 questions or inputs, the answers or outcomes you expect, and a note on how many your system got right. You built the thing being evaluated, so now write the test that proves it works. Consider it a preview of where the program goes next.

How we'll review it

We're looking for working software you understand, not polish. The rubric, roughly:

Area	**Meets**	**Exceeds**
Runs from README	Clones and runs with the documented steps	Handles bad input gracefully
RAG quality	Retrieves and cites from context	Tuned k, with a threshold for "I don't know"
Agent + tools	Loop works, tools logged, iteration cap in place	Handles tool errors, parallel calls, or HITL
Architecture choice	A named pattern, used correctly	Justified against an alternative
MCP	One tool or resource working	Clean separation, sensible permissions
Multimodal	STT or TTS in the flow	Latency logged and discussed

Area	Meets	Exceeds
Design note + eval	Honest, specific, in your voice	Real numbers and a real failure story